

RESEARCH ARTICLE

Evaluating Ascon Hardware on 7-Series FPGA Devices

ADEL R. ALHARBI¹, AMER ALJAEDI¹, ABDULLAH ALJUHN¹, MOAHD K. ALGHUSON²,
HUSSAIN ALDAWOOD³, AND SAJJAD SHAUKAT JAMAL⁴

¹College of Computing and Information Technology, University of Tabuk, Tabuk 71491, Saudi Arabia

²Department of Industrial Engineering, Faculty of Engineering, University of Tabuk, Tabuk 47713, Saudi Arabia

³Government Affairs, NEOM, Tabuk 49643, Saudi Arabia

⁴Department of Mathematics, College of Science, King Khalid University, Abha 61413, Saudi Arabia

Corresponding authors: Adel R. Alharbi (aalharbi@ut.edu.sa) and Sajjad Shaukat Jamal (shussain@kku.edu.sa)

The authors extend their appreciation to the Deanship of Research and Graduate Studies at University of Tabuk for funding this work through Research no. 0099-1444-S.

ABSTRACT The applications regarding the Internet of Things (IoT) demand lightweight and robust cryptographic solutions to ensure authenticated encryption with associated data (AEAD). Also, the lightweight cryptographic solutions that provide data confidentiality, integrity, and authenticity in a single algorithm are critical. In this regard, the CAESAR and NIST lightweight cryptography (LWC) competitions were concluded in 2019 and recently in 2023, respectively, with Ascon selected as the new LWC standard. Ascon has been evaluated (in the literature) for various characteristics, including some limited efforts on field-programmable gate array (FPGA) devices. A comprehensive evaluation of Ascon's hardware implementation is still needed. Therefore, this work presents a unified hardware implementation of two variants of Ascon, i.e., Ascon-128 and Ascon-128a, to investigate the performance of the AEAD operation on 7-series FPGA devices up to the post-place and route level. For AEAD computations in our work, an iterative design-level approach with a finite-state machine (FSM)-based dedicated controller is employed. The benchmarking results show that Ascon utilizes 1632, 1497, 1904, and 1756 look-up tables on Virtex, Kintex, Spartan, and Artix 7-series FPGA devices, respectively. The operating frequencies on these devices are 335, 331, 309, and 317 MHz, with power consumptions of 239, 236, 219, and 222 mW. Consequently, our evaluation of Ascon demonstrates higher performance in operating frequency on Kintex-7 FPGA.

INDEX TERMS Ascon, lightweight, cryptography, hardware, accelerator, architecture, IoT, FPGA.

I. INTRODUCTION

The rapid evolution of technology and communications has increased the demand for efficient paradigms like the Internet of Things (IoT). The IoT transforms conventional objects into smart entities, enabling multiple connected devices to send, process, and receive data across various locations with minimal human interaction. These technological developments bring significant benefits, including automation, real-time analysis, enhanced productivity, and improved business opportunities [1], [2]. To harness these benefits, the number of IoT-connected devices is growing exponentially

and is projected to reach 27 billion by 2025 [3]. However, this tremendous increase in internet users raises significant security threats.

To protect IoT devices, cryptographic algorithms have been used in the past, such as elliptic curve cryptography (ECC) [4] or advanced encryption standard (AES) [5]. Since the IoT nodes are inherently resource-constrained, deploying compute-intensive algorithms like ECC and AES [6], [7] when thousands of users communicate simultaneously is challenging. Therefore, "lightweight" cryptography (LWC) is required to meet these demand(s), where simple operations such as Exclusive-OR (XOR), rotations, shifting, and substitutions are needed instead of complex mathematical operations such as polynomial divisions or modular

The associate editor coordinating the review of this manuscript and approving it for publication was Mohammad Hossein Moaiyeri¹.

inversions [8], [9]. One of the methods to enhance the efficiency of LWC schemes is using authenticated encryption (AE). The AE schemes and lightweight data encryption mechanism(s) with authentication properties simultaneously provide data security and integrity [10], which is essential for IoT applications.

In recent years, significant efforts have been made to design and research LWC-based AE schemes for IoT-based constrained devices. For example, in 2018, the National Institute of Standards and Technology (NIST) initiated an LWC competition to standardize algorithms suitable for constrained environments [11]. The NIST has received 57 submissions for evaluations to the first round from 25 countries [12]. Only the 32 promising candidates advanced to the second round based on their security properties [13]. The 32 candidates were further narrowed to ten finalists demonstrating superior performance [14]. Recently, in 2023, NIST selected Ascon as the winner of the LWC contest [15]. Note that Ascon is an LWC-based algorithm that provides authenticated encryption with associated data (AEAD). It utilizes a sponge construction to support encryption/decryption and hashing operations. To implement these (encryption/decryption and hashing) operations, Ascon offers two variants, i.e., Ascon-128 and Ascon-128a. Instead of the NIST standardization contest, Ascon was also chosen earlier in 2019 as the primary choice for lightweight authenticated encryption in the final portfolio of the CAESAR competition [16]. Therefore, these recently finalized NIST and CAESAR contests for LWC standardization schemes direct researchers and developers to evaluate the performance of the current standard, i.e., Ascon, on different implementation platforms.

Software and hardware are commonly used platforms to evaluate the performance of the traditional and LWC algorithms [4], [7]. It is evident in the literature that software platforms like microcontrollers offer higher flexibility but limited performance throughput, while hardware implementations, such as field-programmable gate arrays (FPGA) and application-specific integrated circuits (ASICs), provide higher throughput but limited flexibility [17]. There is always a trade-off between flexibility and throughput. Therefore, we prefer FPGA-based hardware implementation of the Ascon standard for our evaluation, as the development cost is significantly lower than ASIC implementations.

Instead of the (software and hardware) implementation platforms, the hardware accelerator for the Ascon lightweight cryptographic standard is specifically designed to enhance security in various applications where power, area, and performance are critical. Some considerations include IoT devices, embedded systems, and mobile technology, where resources are limited. Examples of hardware accelerators of Ascon for these applications are described in [10], [18], [19], and [20]. Moreover, the designs of [21], [22], and [23] reveal that the Ascon hardware accelerator is ideal for securing communications in small-scale devices such as wearable technology, smart home devices, and

TABLE 1. Existing accelerators of Ascon on FPGA platform. LUTs shows the look-up tables. Freq is the operating frequency. TP and TP/A denote the throughput in Mbps and the ratio of throughput with LUTs as an area in Mbps/LUT, respectively.

Designs	Device	LUTs	Freq (MHz)	TP (Mbps)	TP/A (Mbps/LUT)
[10]	Spartan-6	2.06k	206.2	315.2	0.153
[18]	Spartan-6	0.68k	216.0	60.1	0.260
[19]	Spartan-6	1.64k	146.1	114.0	0.070
[24]	Spartan-6	1.91k	174.4	1116.4	0.584
[20]	Virtex-7	2.70k	270.3	721.5	0.125
[25]	Artix-7	1.33k	107.0	457.0	0.343
[26]	Virtex-7	6.53k	104.0	13312	2.036

sensors in industrial and healthcare settings. In addition, the robust encryption capabilities of Ascon make it suitable for emerging applications in the Internet of Medical Things (IoMT) and smart grid infrastructure, where data integrity and security are vital for reliable operation. Regarding these applications, the Ascon hardware designs are discussed in [24] and [25]. In short, these diverse applications demand versatile and efficient hardware accelerators of the Ascon lightweight cryptography standard with low-area footprints, minimum power consumption and reasonable throughput. Hence, state-of-the-art has many efforts to implement Ascon hardware accelerators, further discussed in the upcoming sections.

A. ASCON FPGA-EVALUATED DESIGNS

The existing FPGA-based Ascon hardware accelerators are summarized in Table 1.

To address the diverse requirements of IoT-based devices, ranging from high-throughput to area-constrained, three generic Ascon designs are implemented in [10] based on unrolled, round-based, and serialized architectural approaches. All these Ascon designs are benchmarked on Spartan-6 FPGA devices to evaluate area and timing results, including the performance metrics like product of throughput and area.

In [18], several lightweight AE schemes are evaluated from the CAESAR contest, including the Ascon, where authors have targeted the internal bus size of 32-bit to minimize the area utilization. In addition, a single-port memory stores secret keys, initialization vectors, and intermediate results during state updates. The bit-slice approach implements the S-box, sliced into 64×5 partitions. The state value in memory is updated on a word basis, while essential iterative components are designed using a 64-bit internal data path. The area and timing implementation results are reported on Spartan-6 FPGA. Similar to [18], the work in [19] evaluates two candidates from the CESAR competition, i.e., Ascon and ACORN, where a detailed comparison is presented in terms of area and throughput, using the advanced encryption standard-Galois/counter mode (AES-GCM) as a reference. The evaluations are benchmarked on Spartan-6 FPGA devices.

An iterative Ascon implementation is described in [24], where authors proposed an implementation strategy to

TABLE 2. Existing ASIC-specific hardware accelerators of Ascon. TP denotes the throughput in Mbps. The area is reported in either gate equivalents (GE) or μm^2 .

Designs	Technology	Area	TP (Mbps)	Energy	Power (μW)
[21] [†]	90nm	2.57 kGE	384	5706 $\mu\text{J}/\text{byte}$	15
[21] [‡]		7.08 kGE	5524	33 $\mu\text{J}/\text{byte}$	43
[22]	90nm	10.61 kGE	8425	27 $\mu\text{J}/\text{byte}$	72
[23]	28nm	4970.8 μm^2	-	1079 pJ	745

[†] means the reference design is optimized for the low area.

[‡] denotes the reference design is optimized for the high throughput.

synchronize the number of permutations to the clock cycle utilization so that the number of clock cycles and permutations become equal. In other words, the reference design reduces the clock cycles to optimize the throughput. The evaluated results are benchmarked on a Spartan-6 FPGA device. The data encryption for the AI-enabled IoT devices by employing Ascon has been demonstrated in [20], where the area and timing evaluations are provided on Xilinx Virtex-7 FPGA.

An FPGA design of Ascon for emerging medical-related IoMT devices is described in [25]. This design implements a LUT6-based substitution box as part of the cipher permutation block. The authors use electrocardiogram (ECG) signals, which are electrical signals that show the heart beating, to process medical data and perform encryption and decryption. The results are benchmarked on an Artix-7 device, utilizing 1330 look-up tables, operating at 107 MHz, and consuming power of 3 mW (for 5-bit substitution box), 17 mW (for permutation computation) and 31 mW (for complete design).

An impressive Ascon design has recently been described in [26] where the IPsec Ascon core is implemented on a Virtex-7 FPGA board to secure high-throughput, low-latency IoT networks. The design utilizes 6536 look-up tables as an FPGA area, operates on 104 MHz, and achieves a more than 13 Gbps throughput.

B. ASCON ASIC-SPECIFIC EVALUATIONS

Likewise, the FPGA designs of Ascon in section I-A, the ASIC-optimized accelerators are summarized in Table 2.

The work in [21] describes two architectures, one specific to low-area and another for high-throughput. The low area is achieved by processing one bit at a time in one clock cycle, with overall clock cycles overhead. In their high-throughput architecture, one round of permutation is performed in one clock cycle, reducing the cycle counts overall. Similar area reduction approaches are adopted for hardware implementation of Ascon in [22], resulting in a design suitable for various applications, including wireless sensor nodes, radio-frequency identification network (RFID) tags and embedded systems. An efficient hardware implementation of Ascon is presented in [23] to enable fast recovery from unplanned power failures in IoT environments. This implementation

is employed explicitly for energy harvesting applications to achieve efficient and reliable functionality.

The most recent ASIC-optimized Ascon implementation is reported in [27], where different architectural techniques like loop folding, loop unrolled and fully unrolled are employed. Their fully unrolled architecture demonstrates the highest throughput with area overhead. The loop unrolled Ascon architecture results in a lower area with throughput overhead.

In [28], a comprehensive study of various hardware implementations for Ascon is detailed, encompassing both encryption + tag generation and decryption + tag verification for Ascon's AEAD, as well as its hash function. In addition, this work introduces implementations that include side channel protection through threshold countermeasures and fault protection via triplication and majority-based techniques. Notably, these protective measures are orthogonal, allowing each to be activated or deactivated independently without influencing the other. Performance evaluations of these implementations have been benchmarked on both ASIC and FPGA platforms.

C. OBJECTIVE, CONTRIBUTIONS, AND SIGNIFICANCE

Before defining the objective, contributions, and significance (of this work), it is essential to highlight the research gaps specific to Ascon hardware, as outlined below.

1. The dedicated architectures of the Ascon-128 variant are frequently implemented on FPGA platforms. The related designs are described in [10], [19], [20], and [25]. However, these dedicated architectures are not versatile for diverse cryptographic applications, highlighting the necessity for unified/scalable Ascon architectures. To overcome this, we have implemented an Ascon architecture that supports both the 128 and 128a variants.
2. To the author's best knowledge, limited unified Ascon architectures are implemented on FPGA, as shown in [18], [24], and [28]. These unified architectures differ in various prospects; the design in [18] supports 128 and 128a Ascon variants, while the work in [24] implements Ascon-128 and Ascon-Hash variants. Similarly, [28] features a unified design accommodating the Ascon suite, including the 128, 128a, and hash variants. Indeed, unified designs typically optimize hardware resources at the expense of performance. For example, an Ascon architecture described in [28] achieves a maximum operating frequency of 181 MHz, which may not be adequate for high-speed applications. This highlights the need for high-speed unified Ascon architectures that prioritize performance in operating frequency.
3. Section I-A illustrates that FPGA-based dedicated and unified Ascon implementations are commonly reported for Spartan-6 devices [10], [18], [19]. According to the author's best knowledge, the performance evaluation of a unified Ascon design, which supports the 128 and 128a variants, has never been accessed (before) on

various 7-series FPGA devices. This observation further highlights a significant gap in the existing Ascon evaluation studies.

To address the abovementioned gaps, we aim to evaluate the current NIST LWC standard, Ascon, for applications demanding low-area footprints with minimum power consumption and reasonable throughput. Our contributions are given below.

1. *Detailed Ascon descriptions for implementation:* We have provided comprehensive details to implement the current LWC standard, recommended by NIST, i.e. Ascon, in section II.
2. *Unified Ascon architecture:* A novel unified architecture for two variants of Ascon, (i) Ascon-128 and (ii) Ascon-128a, is implemented to perform the authenticated encryption with associated data using one permutation per clock cycle, see section III. Moreover, in our unified architecture, a finite-state-machine (FSM)-based dedicated controller is implemented to facilitate the one permutation per clock cycle, see section III-F.
3. *Design optimizations:* We have efficiently re-utilized the hardware resources to keep minimum hardware area. Moreover, we have used data buffers in the proposed design's datapath to maximize the circuit frequency of the proposed unified architecture.
4. *Performance evaluations:* We have evaluated our unified Ascon architecture comprehensively regarding hardware utilization, clock cycles, the computation time for encryption + tag generation and decryption + tag verification, and the consumed power. Also, figure-of-merits (FoMs) are defined to access area and computation time simultaneously (see section IV).

We have implemented our unified architecture of Ascon-128 and Ascon-128a variants in Verilog HDL using Vivado 2018. We benchmark our results after the post-place and route level on 7-series FPGA devices (Virtex, Kintex, Spartan, and Artix 7). Our unified Ascon design utilizes 1632, 1497, 1904, and 1756 look-up tables on Virtex, Kintex, Spartan, and Artix 7-series FPGA devices, respectively. The achieved circuit frequencies on these FPGA devices are 335, 331, 309, and 317 MHz, with power consumptions of 239, 236, 219, and 222 mW. Additionally, our Ascon design demonstrates efficiency in operating frequency and comparable hardware area utilization with existing FPGA-based benchmarks from the literature. The implementation results show the suitability of our unified Ascon design for broad cryptographic applications, including IoT, embedded systems, mobile technologies, and IoMT. Note that depending on the targeted application, minor changes in the interface of the proposed Ascon design may be required, but overall, it is suitable for secure authentication and efficient data encryption and decryption.

The rest of the paper is organized as follows. The mathematical background of the Ascon is described in Section II. The proposed architecture of Ascon is discussed in Section III. The implementation results and comparison

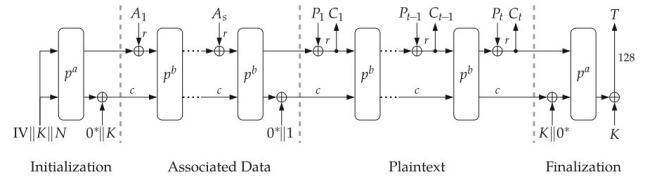


FIGURE 1. Flow chart of the Ascon encryption operation [29].

are discussed in Section IV. Finally, section V concludes the paper.

II. MATHEMATICAL BACKGROUND

As previously mentioned, Ascon supports both AEAD and hashing operations. This work focuses solely on the AEAD operation, excluding hash computation. Also, as reported earlier, Ascon has two variants: (i) Ascon-128 and (ii) Ascon-128a. The primary difference between these variants lies in the size of the data block processed and the number of inner rounds required for permutations, detailed further in section II-A. It is important to note that both Ascon variants offer 128-bit security and use the same 320-bit permutation with different round numbers. The AEAD operation includes encryption + tag generation, described in section II-B, and decryption + tag verification, described in section II-C.

A. ASCON PARAMETERS

The implementation parameters of the AEAD for Ascon variants, i.e., Ascon-128 and Ascon-128a, are provided in Table 3. Column one shows the variant, and the name of the corresponding encryption and decryption algorithm is given in column two. The encryption algorithm is denoted with $\mathcal{E}_{k,r,a,b}$. Similarly, the decryption algorithm is presented with $\mathcal{D}_{k,r,a,b}$. Columns three to six provide the bit size of the secret key, Nonce, tag and data block. Note that the Nonce is the public message number, and the tag is the unique 128-bit binary string for authentication purposes. Finally, the last two columns show the permutation rounds needed to implement the Ascon, where p denotes the permutation, while a and b show the internal round numbers.

B. ENCRYPTION + TAG GENERATION

Encryption transforms original data into a different format, making it incomprehensible to attackers. The inputs and outputs of the Ascon encryption block are detailed in Eq. 1.

$$\underbrace{\mathcal{E}_{k,r,a,b}(K, N, A, P)}_{\text{inputs}} = \underbrace{(C, T)}_{\text{outputs}} \quad (1)$$

The flow chart of the authenticated encryption process for the Ascon lightweight suite is illustrated in Fig. 1. The encryption procedure, denoted as $\mathcal{E}_{k,r,a,b}$, takes several inputs: a 128-bit secret key K , a 128-bit nonce N , associated data A of arbitrary length, and a plaintext P of arbitrary size. The output produced is the authenticated ciphertext C , which has the same length as the plaintext P , along with

TABLE 3. Ascon parameters. More details can be found in [29].

Name	Algorithms	Bit size of				Rounds	
		Key (k)	Nonce	Tag	Data Block (r)	p^a	p^b
Ascon-128	$\mathcal{E}_{k,r,a,b}, \mathcal{D}_{k,r,a,b}$	128	128	128	64	12	6
Ascon-128a	$\mathcal{E}_{k,r,a,b}, \mathcal{D}_{k,r,a,b}$	128	128	128	128	12	8

a 128-bit authentication tag T that authenticates both the associated data and the encrypted message. The mathematical formulation of the Ascon encryption operation is also provided in Algorithm 1.

Algorithm 1 Ascon Encryption Algorithm. More Details Are in the Specification Document [29]

Require: $key K \in \{0, 1\}^k, k \leq 160, N \in \{0, 1\}^{128}, A \in \{0, 1\}^*, P \in \{0, 1\}^*, T \in \{0, 1\}^{128}$

Ensure: $C \in \{0, 1\}^{|P|}, T \in \{0, 1\}^{128}$

▷ **Step#1: Initialization**

1: $S \leftarrow IV_k, r, a, b \parallel K \parallel N$

2: $S \leftarrow p^a(S) \oplus (0^{320-k} \parallel K)$

▷ **Step#2: Processing Associated Data**

3: **if** ($|A| > 0$) **then**

4: $A_1 \dots A_s \leftarrow r\text{-bit blocks of } A \parallel 1 \parallel 0^*$

5: **for** $i = 1, \dots, s$ **do**

6: $S \leftarrow p^b((S_r \oplus A_i) \parallel S_c)$

7: **end for**

8: **end if**

9: $S \leftarrow S \oplus (0^{319} \parallel 1)$

▷ **Step#3: Processing Plaintext**

10: $P_1 \dots P_t \leftarrow r\text{-bit blocks of } P \parallel 1 \parallel 0^*$

11: **for** $i = 1, \dots, t-1$ **do**

12: $S_r \leftarrow S_r \oplus P_i$

13: $C_i \leftarrow S_r$

14: $S \leftarrow p^b(S)$

15: **end for**

16: $S_r \leftarrow S_r \oplus P_t$

17: $\tilde{C}_t \leftarrow \lfloor S_r \rfloor_{|P| \bmod r}$

▷ **Step#4: Finalization**

18: $S \leftarrow p^a(S \oplus (0^r \parallel K \parallel 0^{320-r-k}))$

19: $T \leftarrow \lceil S \rceil^{128} \oplus \lceil K \rceil^{128}$

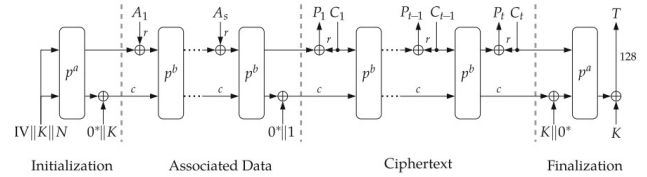
20: **return** $C_1 \dots C_{t-1} \parallel \tilde{C}_t, T$

C. DECRYPTION + TAG VERIFICATION

Decryption is the reverse of encryption block. It transforms the data back into its original format. The inputs/outputs to/from the Ascon decryption block are shown in Fig. 2.

$$\underbrace{\mathcal{D}_{k,r,a,b}(K, N, A, C, T)}_{\text{inputs}} \in \underbrace{(P, \perp)}_{\text{outputs}} \quad (2)$$

Similar to the encryption procedure, the flow chart of the decryption block of the Ascon lightweight suite is shown in Fig. 2. The decryption procedure, denoted as $\mathcal{D}_{k,r,a,b}$, takes as inputs a 128-bit secret key K , a 128-bit nonce N , associated

**FIGURE 2.** Flow chart of the Ascon decryption operation [29].

Algorithm 2 Ascon Decryption Algorithm. More Details Are in the Specification Document [29]

Require: $key K \in \{0, 1\}^k, k \leq 160, N \in \{0, 1\}^{128}, A \in \{0, 1\}^*, C \in \{0, 1\}^*, T \in \{0, 1\}^{128}$

Ensure: $P \in \{0, 1\}^{|C|}$ or $\perp$

▷ **Step#1: Initialization**

1: $S \leftarrow IV_k, r, a, b \parallel K \parallel N$

2: $S \leftarrow p^a(S) \oplus (0^{320-k} \parallel K)$

▷ **Step#2: Processing Associated Data**

3: **if** ($|A| > 0$) **then**

4: $A_1 \dots A_s \leftarrow r\text{-bit blocks of } A \parallel 1 \parallel 0^*$

5: **for** $i = 1, \dots, s$ **do**

6: $S \leftarrow p^b((S_r \oplus A_i) \parallel S_c)$

7: **end for**

8: **end if**

9: $S \leftarrow S \oplus (0^{319} \parallel 1)$

▷ **Step#3: Processing Ciphertext**

10: $C_1 \dots C_{t-1} \tilde{C}_t \leftarrow r\text{-bit blocks of } C, 0 \leq |C_t| < r$

11: **for** $i = 1, \dots, t-1$ **do**

12: $P_i \leftarrow S_r \oplus C_i$

13: $S \leftarrow C_i \parallel S_c$

14: $S \leftarrow p^b(S)$

15: **end for**

16: $\tilde{P}_t \leftarrow \lfloor S_r \rfloor_{|C_t|} \oplus \tilde{C}_t$

17: $S_r \leftarrow S_r \oplus (P_t \parallel 1 \parallel 0^*)$

▷ **Step#4: Finalization**

18: $S \leftarrow p^a(S \oplus (0^r \parallel K \parallel 0^{320-r-k}))$

19: $T^* \leftarrow \lceil S \rceil^{128} \oplus \lceil K \rceil^{128}$

20: **if** $T = T^*$ **then**

21: **return** $P_1 \dots P_{t-1} \parallel \tilde{P}_t$

22: **end if**

23: **else return** $\perp$

data A of arbitrary length, and a ciphertext C of arbitrary length. It produces an output of either the plaintext P (if the tag verification is correct) or an error $\perp$ (if the tag verification fails). The mathematical statements of the Ascon decryption operation are provided in Algorithm 2.

III. PROPOSED ASCON HARDWARE ARCHITECTURE

Before discussing our proposed unified Ascon design, we outline critical design choices. The top-level interface, which manages inputs and outputs, often varies in bit length to suit different applications. Following the CAESAR project team's recommendation, we adopt a standard 32-bit interface for our design [16]. Memory is a pivotal component at the design level, which stores input and output data. The inherent access times of block RAMs for FPGA and SRAMs for ASIC memories can extend the critical path delay, thus lowering the operating frequency of the design [30]. This aspect is further detailed in section III-A. Instead of traditional memory, we use input and output buffers. Another vital design element is the permutation block. Various strategies are employed to optimize the Ascon permutation as evidenced in hardware designs from [10], [27], and [28]. We chose an iterative approach, which, while more straightforward, helps minimize hardware resources and power consumption. With this in mind, our efforts focus on optimizing the iterative approach at the design level to improve the design's circuit frequency for high speed. This is obtained using the buffers in the datapath of the proposed unified Ascon design.

The block diagram of our proposed Ascon design to compute encryption + tag generation and decryption + tag verification is shown in Fig. 3. It comprises several building blocks, such as input controller, output controller, input buffers, output buffers, Ascon-controller and Ascon-permutation. We will provide the implementation details of these blocks in the upcoming sections. The clock, reset, start and done signals are not shown for simplicity. Only 32-bit input 'din' and output 'dout' signals are shown. Let us discuss the building blocks of the design below.

A. INPUT AND OUTPUT CONTROLLERS

As shown in Eq. 1, Eq. 2, Algorithm 1, and Algorithm 2, the inputs and outputs of the Ascon encryption and decryption blocks include a key, nonce, associated data, plaintext/ciphertext, and tag. Therefore, our Ascon encryption and decryption architecture, illustrated in Fig. 2, consists of five 128-bit input buffers (indicated by a blue box) and two 128-bit output buffers (indicated by a green box). The five input buffers are K (secret key), N (nonce), AD (associated data), PCT_in (plaintext/ciphertext as input), and T_in (tag as input). The (two) output buffers are PCT_out (plaintext/ciphertext as output) and T_out (tag as output). The input buffers hold the initial values for the encryption-decryption operations, while the output buffers store the results after computations.

As discussed in the previous text, the input and output buffers keep the initial and final outputs, respectively. Despite using the buffers, it is also possible to use instantiated memories like block RAM for FPGAs and compiled memories like SRAM for ASIC demonstrations. Notably, instantiated and compiled memories have associated access time for reading and writing data, resulting in decreased

circuit frequency. This is experimentally shown (before) in [30] for one of the post-quantum SABER protocols on 65nm ASIC technology. In other words, the longer access time for reading and writing data from/to memory blocks also affects the critical path. For example, an iterative and unified Ascon design of [23] uses a single-port memory and achieves an operating frequency of 216 MHz on Spartan-6 FPGA. In contrast, our iterative and unified Ascon design incorporates the buffers to keep the initial and final outputs and achieves the maximum circuit frequency of 309 MHz on the Spartan-7 device. Therefore, our buffer approach helps in optimizing the operating frequency. This will be further explained later in the results section.

To populate the input and output buffers, FSM-based controllers, referred to as the input and output controllers, are used as shown in Fig. 3. The input controller sequentially receives 32-bit input values (from) outside the core and determines which input buffer (K, N, AD, PCT_in, or T_in) to fill. Similarly, the output FSM-based controller takes 128-bit data from the corresponding output buffers (PCT_out and T_out) and outputs it sequentially in 32-bit segments through the Ascon output pin.

It is essential to note that a 32-bit counter register is used to load data in 32-bit format into the 128-bit input buffers, although it is not shown in Fig. 3 but is a part of the design. This same counter register is also reused to control loading data in 32-bit format from the 128-bit output buffers to the Ascon design output pin. Considering the reuse strategies like the counter register in our design helps us to keep the minimum hardware resources.

B. INITIALIZATION VECTOR (IV)

As provided earlier in the first paragraph of section II, Ascon has two variants: Ascon-128 and Ascon-128a. The distinct implementation parameters differentiate these Ascon variants. The Ascon specification document [29] uses an initialization vector (IV) to hold the parameters needed to initiate the implementation of Algorithm 1 and Algorithm 2, as indicated in the first line of these algorithms. The format of the IV is shown in Eq. 3.

$$\begin{aligned}
 IV_{k,r,a,b} &\leftarrow k \parallel r \parallel a \parallel b \parallel 0^{160-k} \\
 &= \underbrace{80400c0600000000}_{\text{for Ascon-128}} \\
 &\quad \times \underbrace{80800c0800000000}_{\text{for Ascon-128a}}
 \end{aligned} \tag{3}$$

Eq. 3 specifies the algorithm parameters, including the key size k , the rate r , the initialization and finalization round number a , and the intermediate round number b , each written as an 8-bit integer (for more details, see [29]). The dedicated designs of [10], [19], [20], and [25] support the Ascon-128 variant, which requires only a 64-bit buffer to hold parameters from Eq. 3 to initiate the Ascon implementation. Comparatively, in our proposed architecture, shown in Fig. 3, we utilize a 128-bit buffer named IV, which contains the

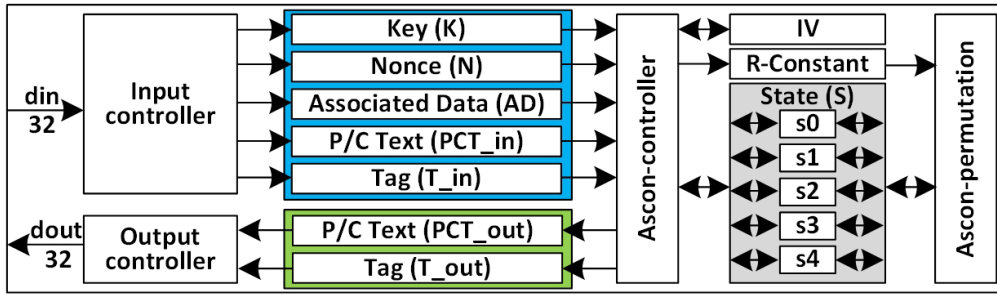


FIGURE 3. Block diagram of the proposed iterative Ascon architecture.

values of the corresponding parameters from Eq. 3 for the subsequent implementation of the Ascon variant, i.e., Ascon-128 or Ascon-128a.

The reason to use a 128-bit buffer for IV is because of implementing Ascon-128 and Ascon-128a variants in a single design. In other words, the data block for Ascon-128 is 64 bits, while the data block for Ascon-128a is 128 bits. The size of the data blocks is denoted before in Table 3 in section II-A. Therefore, we utilize a 128-bit buffer instead of a 64 bit. The Ascon controller sets the IV buffer with the corresponding values, as illustrated in Fig. 3. Moreover, choosing the IV value to implement Ascon-128 or Ascon-128a variants depends on a one-bit ‘mode’ signal, not shown in Fig. 3. Whenever the ‘mode’ signal becomes ‘0’, the IV is initialized with parameters of Ascon-128. Otherwise, the IV is initialized with parameters of Ascon-128a.

C. ROUND CONSTANTS BLOCK (R-CONSTANT)

Implementing Algorithm 1 and Algorithm 2 requires the number of permutations p^a and p^b , and the corresponding values of p^a and p^b are 12, 8, and 6, as shown (before) in Table 3 in section II-A. Moreover, to implement encryption and decryption algorithms of Algorithm 1 and Algorithm 2, p^a and p^b require round constant values, detailed in columns one to three and five to seven of Table 4. In our architecture to implement Table 4, we implemented a combinatorial block, R-Constant, which takes an address as input and outputs a 64-bit constant value to compute the corresponding permutations p^{12} , p^8 , and p^6 of Algorithm 1 and Algorithm 2. In other words, the Ascon controller generates the address which feeds as input to the R-Constant block, and the output of the R-Constant block feeds as input to the permutation block, as shown in Fig. 3.

D. ASCON STATE (S)

The Ascon has a structure called Substitution-Permutation-Network (SPN). Therefore, the permutations in Ascon iteratively apply SPN-based round transformations. The round transformation operations are the addition of round constant, substitution and diffusion. We describe these permutation transformation operations in the next section. However, remember that a 320-bit state S is required to implement

the operations for round transformations. Therefore, in this section, we describe only the implementation of the Ascon state S . The state S is also named S-box in Ascon, but in this work, we called it (simply) by state, as shown in Fig. 3.

To implement the round transformation operations of the addition of round constant, substitution and diffusion, we used an array of register with size 320-bit S in Fig. 3, grey filled box. The 320-bit S is split into five 64-bit register words, i.e., $s0$, $s1$, $s2$, $s3$ and $s4$. Each 64-bit word receives/outs 64-bit value from/to the Ascon controller and the permutation blocks in Fig. 3. This communication is denoted with double arrows in Fig. 3. The concatenation of the 64-bit word registers, i.e., $s0 \parallel s1 \parallel s2 \parallel s3 \parallel s4$, formulates the state S .

To implement the Algorithm 1 and Algorithm 2, the initial value to the state S is $IV_{k,r,a,b} \parallel K \parallel N$, where $\parallel$ denotes the concatenation. This initial setting to S is presented in line one of Algorithm 1 and Algorithm 2. It can be observed from Algorithm 1 and Algorithm 2 that all the Ascon operations, including the permutations p^a and p^b , perform using S . Therefore, in our architecture, we re-utilized one buffer of 320-bit state S to operate encryption and decryption operations of Algorithm 1 and Algorithm 2 iteratively. This re-utilization of S in an iterative way helps us to keep minimum hardware resources in our proposed architecture of Fig. 3. In contrast, the most recent Ascon implementation of [27] utilizes multiple state buffers in the datapath to reduce clock cycles with hardware resource overhead. Note that there is always a trade-off.

E. ASCON PERMUTATION

The permutation is the critical building block of the Ascon. As hardware accelerator, there are two possibilities to implement the Ascon permutation: (i) using an iterative approach and (ii) employing multiple permutation blocks depending on the number of rounds to reduce clock cycles. The first approach is previously utilized in Ascon designs of [18], [24], [25], and [28], and is also used in our work to minimize the hardware resources and the consumed power. The second approach has recently been employed in Ascon accelerator(s) of [27] to optimize the throughput. Hence, in the following text, we discuss only

TABLE 4. Ascon round constants c_r used in each round i of p^a and p^b . See [29] for more details.

p^{12}	p^8	p^6	Constant Values (c_r)	p^{12}	p^8	p^6	Constant Values (c_r)
0	–	–	00000000000000f0	6	2	0	0000000000000096
1	–	–	00000000000000e1	7	3	1	0000000000000087
2	–	–	00000000000000d2	8	4	2	0000000000000078
3	–	–	00000000000000c3	9	5	3	0000000000000069
4	0	–	00000000000000b4	10	6	4	000000000000005a
5	1	–	00000000000000a5	11	7	5	000000000000004b

the implementation of Ascon permutation using an iterative approach.

As shown in Fig. 3, the permutation block takes 320-bit data from state S as input and produces 320-bit data to update the state S as output. The internal architecture of the permutation block comprises three units, not shown in Fig. 3, (i) the addition of round constants, (ii) substitution and (iii) diffusion operations. The implementation details of these units are given below.

1) ADDITION OF ROUND CONSTANTS

As shown in Fig. 3, the permutation block takes a 64-bit round constant value from the R-Constant block. Therefore, the round constant step adds a 64-bit round constant value from the R-Constant block to register word s_2 of the state S in round i , where i goes from 1 to 12, 8 or 6 for corresponding p^{12} , p^8 or p^6 . The corresponding operation is shown in Eq. 4.

$$s_2 \leftarrow s_2 \oplus c_r \quad (4)$$

In Eq. 4, c_r is a 64-bit round constant value from the R-Constant block. We want to highlight that the Ascon specification document in [29] specifies the values of i and r where to compute p^a , $i = r$, while to compute p^b , $r = i + a - b$. In this manuscript, see the corresponding entries regarding p^{12} , p^8 and p^6 in Table 4. It can be observed from Eq. 4 that adding a round constant needs an array of 64 XORs for hardware implementations (in an iterative way).

2) SUBSTITUTION

Generally, substitution is a method in which each letter in the plaintext is replaced by another letter or symbol according to a secret key. Therefore, two approaches are available to implement the substitution block for Ascon: (i) using a look-up table (LUT)-based and (ii) using the coordinate functions. The LUT-based implementation is more expensive or uses significant hardware resources, as on 7-series FPGA devices, one LUT equals 100 slices [31]. On the other hand, the coordinate function-based implementation takes much less area than the LUT-based implementation [32]. Thus, in our hardware architecture, we implemented the coordinate-based implementation of the Ascon substitution, which is mathematically expressed in Eq. 5, as shown at the bottom of the next page, where $(\oplus)$ and $(\cdot)$ show the bitwise XOR and AND operations, respectively. Therefore, in our work, we implemented the Ascon substitution using Eq. 5, which requires only the bitwise XOR and AND

operations. For more mathematical details of the substitution implementation, we refer to consult [29] and [32].

3) DIFFUSION

The diffusion operation provides diffusion within each 64-bit register word s_i , where $0 \leq i \leq 4$. It applies a linear function $\sum_i(s_i)$ defined in Eq. 6 to each word s_i , where the $(\oplus)$ is a bitwise XOR operation and $(\ggg)$ is the rotation towards the right (or a circular shift). Hence, in this work, we implemented the Ascon diffusion using Eq. 6, as shown at the bottom of the next page, and it needs only the bitwise XOR and circular shift operations. We recommend to follow [29] and [32] for more details.

F. ASCON CONTROLLER

The Ascon controller in our proposed architecture ensures the correct implementation of the four steps, i.e., (i) initialization, (ii) processing associated data, (iii) processing plain/cipher text, and (iv) finalization, of the Algorithm 1 and Algorithm 2. We provide the implementation details of these steps below.

1) INITIALIZATION (LINES 1 AND 2 OF ALGORITHM 1 AND ALGORITHM 2)

Line 1 in Algorithm 1 and Algorithm 2 sets the initial values of the Ascon state S with an IV concatenated with secret key K and a nonce. The $\parallel$ denotes the concatenation. Setting the IV is already described in section III-B, which mainly depends on a one-bit ‘mode’ signal. The setting of the state S takes 1 clock cycle. After setting the state S , the 12 rounds of the permutation p^a will be performed in 12 clock cycles on the initial state S followed by an XOR (i.e., $\oplus$) operation with concatenated bit string of $0^{320-k} \parallel K$. This process is illustrated in line 2 of Algorithm 1 and Algorithm 2. A total of 13 clock cycles are needed to implement the initialization step of the Ascon-128 and Ascon-128a encryption and decryption algorithms.

2) PROCESSING ASSOCIATED DATA (LINES 3 TO 9 OF ALGORITHM 1 AND ALGORITHM 2)

Line 3 of the algorithms checks the length of the associated data; if it is higher than 0, then line 4 makes r -bit chunks of the associated data, where r denotes the data block. The last chunk must be produced with the concatenation of 1 and a string of all zeros (0^*). After that, permutation of p^b will be performed iteratively on each associated data segment A_i XORed with the current state S_r and concatenated with

S_c . This process is denoted in a *for* loop in lines 4 to 6 of Algorithm 1 and Algorithm 2. When the last associated data segment is processed, the current state S will be XORed with a bit string of all zeros concatenated with 1 (line 9 of the encryption and decryption algorithms). Note that in our Ascon architecture of Fig. 3, we limit the length of associated data to 128 bits, meaning that our design can handle associated data lengths up to 128 bits (only). To handle more than 128 bits of associated data length, a wrapper across the proposed design of Fig. 3 is required. Regarding clock cycles, one p^b permutation needs 12 and 8 cycles (in a fully iterative method) for Ascon-128 and Ascon-128a variants, respectively. Also, two additional clock cycles are needed to compute the XOR and concatenations of lines 6 and 9 of Algorithm 1 and Algorithm 2. Overall, step#2 of the algorithms requires 14 (for Ascon-128) and 10 (for Ascon-128a) clock cycles when a fixed 128-bit of associated data is processed.

3) PROCESSING PLAIN/CIPHER TEXT (LINES 10 TO 17 OF ALGORITHM 1 AND ALGORITHM 2)

The lines 10, 16 and 17 in Algorithm 1 and Algorithm 2 denotes the split of plaintext or ciphertext in r -bit blocks to process, where r denotes the data block of 64-bit for Ascon-128 and 128-bit for Ascon-128a. Note that the proposed design encrypts and decrypts the data using 128-bit size. This means that the plaintext and the ciphertext lengths are 128-bit fixed. This could be made generic to process multiple blocks of the plaintext and ciphertext, but this needs an additional wrapper across the proposed design of Fig. 3. The remaining lines from 11 to 15 of Algorithm 1 and Algorithm 2 are implemented in this work. We need only an XOR operation to produce the ciphertext (during encryption) or plaintext (during decryption); see line 12 of the corresponding algorithms. Line 13 of the Ascon encryption and decryption algorithms update the state S . Then, a permutation of p^b needs to be performed on the updated state S , shown in line 14 of the algorithms. For the Ascon-128 variant, 14 clock cycles are needed to process one 128-bit plaintext or ciphertext; the 12 cycles are for p^b permutation computation, and 2 cycles to compute lines 12 and 13 of the corresponding algorithms. Similarly, for the Ascon-128a variant, 10 clock cycles are

needed to process one 128-bit plaintext or ciphertext; the 8 cycles are for p^b permutation computation, and 2 cycles to compute lines 12 and 13 of the corresponding algorithms.

4) FINALIZATION

In the finalization step of Algorithm 1 and Algorithm 2, one permutation over the state S is needed to perform, which needs 12 clock cycles to compute for both Ascon variants. In the case of encryption (i.e., Algorithm 1), a 128-bit tag T will be produced using XOR of S with key K , which needs one clock cycle to operate. In total, 13 clock cycles are needed for implementing Algorithm 1. In the case of decryption operation (Algorithm 2), the generated tag T^* during the decryption process will be compared with the input tag T ; if both these equals, the decrypted process releases the decrypted text as output. Otherwise, it produces an error. Compared to Algorithm 1, Algorithm 2 requires one additional clock cycle for $T^* = T$ comparison. Hence, the total cycle count for the finalization step of the decryption algorithm is 14.

The Ascon-128 variant needs 55 (13+14+14+14) clock cycles to process one 128-bit data. The Ascon-128a variant needs 47 (13+10+10+14) clock cycles to process one 128-bit data. The clock cycles for the Ascon-128a are lower than the Ascon-128 as the data blocks differ. The data block for Ascon-128 is 64-bit, while the data block of Ascon-128a is 128-bit.

IV. RESULTS AND COMPARISONS

We have implemented the Ascon architecture in Verilog HDL using the Vivado IDE tool. Moreover, the 7-series FPGA devices (Virtex, Kintex, Spartan and Artix) are used for logic synthesis and implementation.

A. IMPLEMENTATION RESULTS

The implementation results after the post-place and route are shown in Table 5. Column one specifies the implementation device, and column two lists the implemented Ascon variant. Columns three to five present the hardware resources regarding slices, LUTs, and FFs. Column six provides information on the clock cycles, while column seven details the implementation frequency in MHz. The time to perform

$$\begin{aligned}
 s_0 &= (s_4 \cdot s_1) \oplus s_3 \oplus (s_2 \cdot s_1) \oplus s_2 \oplus (s_1 \cdot s_0) \oplus s_1 \oplus s_0 \\
 s_1 &= s_4 \oplus (s_2 \cdot s_3) \oplus s_3 \oplus (s_3 \cdot s_1) \oplus s_2 \oplus (s_1 \cdot s_2) \oplus s_1 \oplus s_0 \\
 \text{Substitution} = s_2 &= (s_4 \cdot s_3) \oplus s_4 \oplus s_2 \oplus s_1 \oplus 1 \\
 s_3 &= (s_4 \cdot s_0) \oplus (s_3 \cdot s_0) \oplus s_4 \oplus s_3 \oplus s_2 \oplus s_1 \oplus s_0 \\
 s_4 &= (s_4 \cdot s_1) \oplus s_4 \oplus s_3 \oplus (s_1 \cdot s_0) \oplus s_1
 \end{aligned} \tag{5}$$

$$\begin{aligned}
 s_0 &\leftarrow \sum_0(s_0) = s_0 \oplus (s_0 \ggg 19) \oplus (s_0 \ggg 28) \\
 s_1 &\leftarrow \sum_1(s_1) = s_1 \oplus (s_1 \ggg 61) \oplus (s_1 \ggg 39) \\
 \text{Diffusion} = s_2 &\leftarrow \sum_2(s_2) = s_2 \oplus (s_2 \ggg 1) \oplus (s_2 \ggg 6) \\
 s_3 &\leftarrow \sum_3(s_3) = s_3 \oplus (s_3 \ggg 10) \oplus (s_3 \ggg 17) \\
 s_4 &\leftarrow \sum_4(s_4) = s_4 \oplus (s_4 \ggg 7) \oplus (s_4 \ggg 41)
 \end{aligned} \tag{6}$$

TABLE 5. Implementation results of the proposed Ascon design on 7-series FPGA platform. For evaluations, the associated data, plaintext, and ciphertext lengths are considered to be 128 bits.

Device	Ascon Variant	Slices	LUTs	FFs	Clock Cycles	Frequency (F in MHz)	Time (T in μ s)	Throughput (TP in Mbps)	Power (mW)
Virtex-7 xc7vx690tffg1930-3	Ascon-128 Ascon-128a	348	1632	912	55 47	335	0.16 0.14	400 914	239
Kintex-7 xc7k480tffg1156-3	Ascon-128 Ascon-128a	265	1497	912	55 47	331	0.16 0.14	400 914	236
Spartan-7 xc7k480tffg1156-3	Ascon-128 Ascon-128a	330	1804	912	55 47	309	0.18 0.15	355 853	219
Artix-7 xc7a100tcs324-3	Ascon-128 Ascon-128a	303	1756	912	55 47	317	0.17 0.15	376 853	222

one encryption and decryption operation, referred to as latency, is shown in column eight and is computed using Eq. 7. Throughput (TP) results are presented in column nine, calculated as the reciprocal of latency using Eq. 8. The last column shows the average dynamic power consumption of the Ascon design. We want to highlight that the consumed power is obtained using Value Change Dump (VCD) files.

$$\text{Latency or Time (in } \mu\text{s)} = \frac{\text{Clock Cycles}}{\text{Frequency (MHz)}} \quad (7)$$

$$\text{Throughput (Mbps)} = \frac{\text{Number of bits processed}}{\text{Time (}\mu\text{s)}} \quad (8)$$

As shown in Table 5, our Ascon-128 and Ascon-128a implementations utilize lower FPGA area in slices, LUTs and FFs on Kintex-7 FPGA. The utilized slices, LUTs and FFs are 265, 1497 and 912. The same Ascon design on other devices of Virtex-7, Spartan-7 and Artix-7 utilizes more resources than the Kintex-7 implementation. Note that the proposed design does not use (any single) digital signal processor (DSP)-blocks. Also, the proposed implemented Ascon architecture does not utilize any memory block for the initial, intermediate and final results. We used registers or buffers instead of an initiated FPGA memory block, like MRAM used in the ASIC Ascon implementation of [23]. This causes identical FFs of 912 on Kintex-7, Virtex-7, Spartan-7 and Artix-7 FPGA devices. As shown in Table 5, our Virtex-7 implementation of Ascon utilizes the highest area in slices (i.e., 348) compared to Kintex, Artix and Spartan 7-series FPGA devices.

The computation cost of the implemented Ascon-128 and Ascon-128a variants require 55 and 47 clock cycles. This cost is for one authenticated encryption with associated data. This means that for Ascon-128, the length of the processed plaintext and ciphertext is 64 bits, while the size of the associated data is 128 bits. Similarly, for Ascon-128a, the length of the processed plaintext, ciphertext and associated data is 128 bits. In contrast, the longer plaintext and ciphertext size of 128 bits for Ascon-128a results in lower clock cycles. More clock cycle details are already described in section III-F.

As reported before, the proposed design utilizes registers or buffers for the initial, intermediate, and final results, which increases the operating frequency but with minor area overhead. The highest operating frequency of 335 MHz is

achieved for Virtex-7 FPGA. On the other hand, this highest frequency results in higher design power consumption (i.e., 239 mW). The lowest operating frequency of 309 MHz is achieved for Spartan-7 FPGA, where the power consumption of the Ascon design is also minimum, i.e., 219 mW. The other two implementations on Kintex-7 and Artix-7 devices reveal results comparable to those of Virtex-7 and Spartan-7 FPGA devices, shown in Table 5.

The lower latency or time and higher throughput results are achieved for Virtex-7 and Kintex-7 FPGA devices. In other words, the latency and throughput results on Virtex-7 and Kintex-7 devices are identical, as shown in columns eight and nine of Table 5. The reason is the almost similar operating frequency of 335 MHz (Virtex-7) and 331 MHz (Kintex-7). With this same performance in latency and throughput, the area in slices for Kintex-7 FPGA is 265, and the power consumption is 236 mW; these values make the Kintex-7 implementation more beneficial than the Virtex-7 implementation, where the used slices are 348, and consumed power is 239 mW.

We have marked until now that the Kintex-7 implementation is more efficient in terms of area, computation time, and consumed power than the Virtex-7 implementation. Let us compare the Kintex-7 implementation with Spartan-7 and Artix-7 FPGA implementations. Therefore, the Kintex-7 implementation in the FPGA area (in slices) is $1.24 \times (\frac{330}{265})$ and $1.14 \times (\frac{303}{265})$ more efficient than Spartan-7 and Artix-7 implementations, respectively. On the other hand, the Spartan-7 and Artix-7 implementations are $1.07 \times$ and $1.06 \times$ more efficient in consumed power than that of the Kintex-7 implementation. Using these four rationales of 1.24, 1.14, 1.07 and 1.06, if we calculate the difference, the Kintex-7 implementation is still $0.17 \times (1.24 - 1.07)$ and $0.08 \times (1.14 - 1.06)$ more efficient compared to Spartan-7 and Artix-7 FPGA implementations. This difference demonstrates that the proposed Ascon design outperforms the Kintex-7 FPGA when (individually) comparing the area, time and power results.

1) FOMS IN TP/SLICES AND TP/LUTS

Instead of evaluating the area and throughput parameters individually, we show the combined area and throughput effect by defining the FoMs in terms of the ratio of throughput

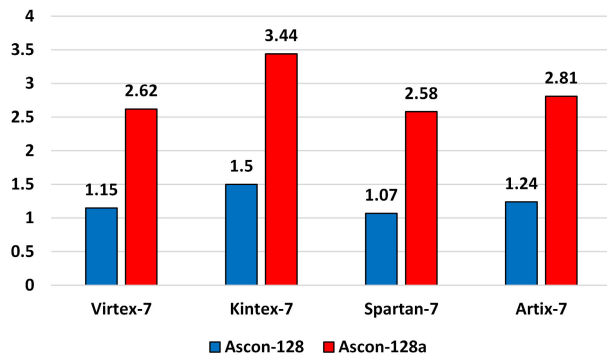


FIGURE 4. FoM in TP (Mbps)/Slices.

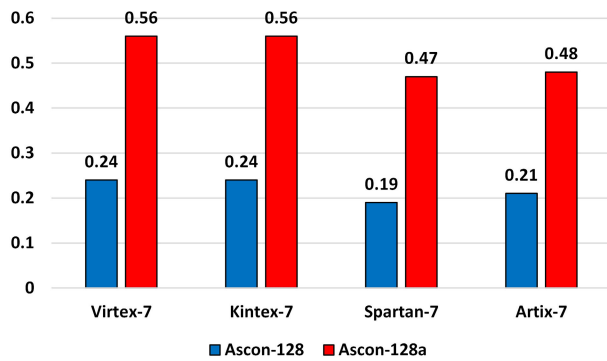


FIGURE 5. FoM in TP (Mbps)/LUTs.

with slices (TP/Slices) and the ratio of throughput with LUTs (TP/LUTs) in Fig. 4 and Fig. 5, respectively. Note that the higher the values of the defined FoMs, the higher will be the performance of the implemented Ascon design. These figures show that the TP/Slices and TP/LUTs for the Ascon-128a variant are higher than those for the Ascon-128 variant. This is because of the 128-bit data block for Ascon-128a, while the Ascon-128 has a data block of 64 bits, shown earlier in Table 3.

Based on the comprehensive details of the implementation results and the calculated FoMs, it can be summarized that the proposed Ascon design is suitable for applications that demand area and power compact implementation with reasonable throughput, such as IoTs and RFID tags.

B. COMPARISONS

We provide the hardware area and operating frequency comparison to the state-of-the-art in Table 6. The other design parameters, like computation time, throughput, and power, can not be compared, as most existing Ascon designs are benchmarked on different implementation platforms and technologies. Therefore, column one provides the designs, while the implementation device is illustrated in column two. The supported Ascon variants are given in column three. From columns four to six, we show the area utilization

of different Ascon designs on FPGA devices in terms of slices, LUTs and FFs. Column seven compares the operating frequency. The last column provides the implemented Ascon architectural details. The ‘-’ in Table 6 means that the corresponding information in the reference design is not reported.

The Ascon implementations of [10] target scalability by executing different numbers of permutations in one clock cycle. However, in Table 6, we provide the implementation details of the reference work for single permutation computation per clock cycle because our proposed unified Ascon design computes one permutation per clock cycle. This will allow us a fair comparison to the reference work. Therefore, in comparison to Spartan-6 implementation of [10], our design implemented on Spartan-7 FPGA utilizes $1.14 \times (\frac{2060}{1804})$ lower LUTs. Similarly, our Spartan-7 implemented design is $1.5 \times (\frac{309}{206})$ more efficient in operating frequency as compared to the Spartan-6 implemented Ascon design of [10]. The reason is different implementation platforms, Spartan-6 in the reference work, while we used Spartan-7 in this work. Another reason is that we use single state S in our implemented Ascon design, while the reference work uses multiple state S buffers for implementations.

On Artix-7 FPGA, the reference work in [24] has implemented six round-2 candidates of the NIST LWC competition, including Ascon. On the same Artix-7 implementation platform, our iterative Ascon implementation is $0.92 \times (\frac{1756}{1898})$ and $1.20 \times (\frac{317}{263})$ more efficient in LUTs and operating frequency. The reason is that the reference design has implemented the Ascon-128 and Ascon-Hash variants, while in this work, we have implemented only the Ascon-128 and Ascon-128a variants without considering the Ascon-Hash.

Similar to the work of [10], a scalable Ascon design is implemented in [20] where authors implemented different permutation rounds per clock cycle. Note that the reported results in Table 6 for the reference work are for single permutation computation per clock cycle. Compared to Virtex-7 implementation of [20], our design implemented on the identical FPGA device utilizes $1.65 \times (\frac{2708}{1632})$ lower hardware resources in LUTs. Also, our Virtex-7 implemented design is $1.24 \times (\frac{335}{270})$ more efficient in operating frequency as compared to the Spartan-6 implemented Ascon design of [20]. We used single state S in our implemented Ascon design, while the reference work uses multiple state S buffers for their implementations, resulting in higher LUTs.

The work in [25] has implemented only the Ascon-128 variant to encrypt and decrypt the ECG-related medical data. On a similar Artix-7 FPGA device, the reference work is $1.32 \times$ and $1.04 \times$ more efficient in LUTs and FFs than our implemented Ascon design, respectively. The reason is that the reference work only supported Ascon-128 while we implemented Ascon-128 and Ascon-128a variants. On the other hand, our design achieves an operating frequency of 317 MHz, which is comparatively $2.96 \times$ higher than

TABLE 6. Comparison to existing Ascon implementations on FPGA platform. The work in [28] utilizes 944 LUTs and 734 FFs for encryption + tag generation. Also, it uses 1058 LUTs and 735 FFs for decryption + tag verification. In the table, we provided the sum of their LUTs and the sum of their FFs for encryption + tag generation + decryption + tag verification. The Ascon-suite means the reference work supports all variants of Ascon, including the hash. A blue checkmark (✓) signifies superior performance compared to existing Ascon accelerators, while a cross mark (×) indicates areas where the performance of our Ascon accelerator falls short. Reference [25] consumes power consumption of 31 mW.

Designs	Device	Variant	Slices	LUTs	FFs	Frequency (MHz)	Architectural Details
[10]	Spartan-6	Ascon-128	–	2060 (✓ 1.14×)	–	206 (✓ 1.5×)	Scalable
[24]	Artix-7	Ascon128 + Ascon-Hash	–	1898 (✓ 0.92×)	–	263 (✓ 1.20×)	Iterative
[20]	Virtex-7	Ascon-128	–	2708 (✓ 1.65×)	–	270 (✓ 1.24×)	Scalable
[25]	Artix-7	Ascon-128	–	1330 (× 1.32×)	870 (× 1.05×)	107 (✓ 2.96×)	Iterative
[18]	Spartan-6	Ascon-128 + Ascon-128a	231 (× 1.42×)	684 (× 2.63×)	268 (× 3.40×)	216 (✓ 1.43×)	Iterative + SP-Memory
[19]	Spartan-6	Ascon-128	–	1640 (× 1.10×)	–	–	DPA-protected
[28]	Kintex-7	Ascon-suite	–	2002 (✓ 1.37×)	1469 (✓ 1.61×)	181 (✓ 1.82×)	Iterative
This Work	Kintex-7	Ascon-128 + Ascon-128a	265	1497	912	331	Iterative
	Virtex-7	Ascon-128 + Ascon-128a	348	1632	912	335	
	Spartan-7	Ascon-128 + Ascon-128a	330	1804	912	309	
	Artix-7	Ascon-128 + Ascon-128a	303	1756	912	317	

the reference design, where the achieved circuit frequency is 107 MHz. This is probably because of the internal buffers, shown in blue and green colours in Fig. 3, in our design to keep the initial and intermediate results. The dynamic power consumption of the reference design is 31 mW, while our architecture utilizes an average power consumption for encryption and decryption, including the tag generation and verification, of 222 mW. Comparatively, the reference design is $7.16 \times (\frac{222}{31})$ more efficient in consumed power than our work. This is because our work operates on a higher circuit frequency than the reference work.

The work in [18] has provided results on Spartan-6 implementation. Therefore, we compared our Spartan-7 results with [18] in this work. Comparatively, for implementing the Ascon-128 and Ascon-128a variants, our implementation on modern Spartan-7 FPGA utilizes higher hardware resources in slices LUTs and FFs than the reference work implemented on Spartan-6, show the corresponding columns in Table 6. The reason is that one single-port memory is used in the reference work to store the key along with state initialization constants. In contrast, we have not opted for (any) memory in our iterative Ascon implementation; instead, we used buffers inside the design.

The work in [19] has highlighted that the Ascon implementations with full-width datapaths, i.e., 64-bit block size in the case of Ascon-128, and basic-iterative architectures (i.e., one clock cycle per round), are not ideal for protection against differential-power-analysis (DPA)-attacks. Generally, differential power analysis is a powerful tool attackers use to extract secret keys and compromise the security of tamper-resistant devices. In other words, the DPA is a type of side-channel attack where statistical analysis is needed to leak the secret information. A comprehensive comparison study of the cost of protection against DPA of several authenticated ciphers is presented in [33]. In comparison, to [19], our one clock cycle per round implementation of Ascon over Spartan-7 utilizes 1804 LUTs, which is comparatively $1.1 \times (\frac{1804}{1640})$ higher than the DPA-protected Ascon implementation of [19] on Spartan-6 FPGA. This is because the reference work has implemented only the Ascon-128 variant, while we

provided implementation for both the Ascon-128 and Ascon-128a variants. Although the data path of 64-bit supporting variants, i.e., Ascon-128 and Ascon-128a, is common in a single design, some buffers are still needed, which infers higher resources in our work.

As far as the Kintex-7 implementation is concerned for comparison, our Ascon implementation is $1.37 \times$ and $1.82 \times$ more efficient in LUTs and circuit frequency. This is because of the Ascon-128 and Ascon-128a variants in our implementation, while the reference design of [28] supports the hash functions of Ascon as well along with the Ascon-128 and Ascon-128a variants.

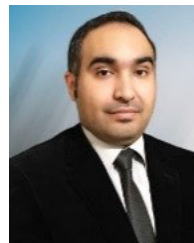
V. CONCLUSION

This paper has presented a unified architecture of Ascon-128 and Ascon-128a variants to compute the AEAD operation for encryption + tag generation and decryption + tag verification. An iterative approach is utilized in this work to process a single permutation per clock cycle. The implementation results after the post-place-and-route level on 7-series FPGA devices (Virtex, Kintex, Spartan, and Artix 7) are presented. The Ascon utilizes 1632, 1497, 1904, and 1756 LUTs on Virtex, Kintex, Spartan, and Artix 7-series FPGA devices, respectively. The achieved circuit frequencies on these FPGA devices are 335, 331, 309, and 317 MHz, with power consumptions of 239, 236, 219, and 222 mW. Although this work reports results using a single permutation per clock cycle on different 7-series FPGA devices, it can be optimized for high-throughput computing multiple permutations per clock cycle like the Ascon demonstrated designs of [10] and [20]. Also, the hashing Ascon variant is not considered for evaluations in this work and could be merged with it in the future without any significant increase in hardware resources.

REFERENCES

- [1] A. Aljuhani, P. Kumar, R. Kumar, A. Jolfaci, and A. K. M. N. Islam, "Fog intelligence for secure smart villages: Architecture, and future challenges," *IEEE Consum. Electron. Mag.*, vol. 12, no. 5, pp. 1–9, Jul. 2022.
- [2] B. Alotaibi and M. Alotaibi, "A stacked deep learning approach for IoT cyberattack detection," *J. Sensors*, vol. 2020, pp. 1–10, Sep. 2020.

- [3] S. Sinha. (2023). *State of IoT 2023: Number of Connected IoT Devices Growing 16% To 16.7 Billion Globally*. Accessed: Jul. 24, 2024. [Online]. Available: <https://iot-analytics.com/number-connected-iot-devices/>
- [4] M. Rashid, M. Imran, A. R. Jafri, and T. F. Al-Somani, "Flexible architectures for cryptographic algorithms—A systematic literature review," *J. Circuits, Syst. Comput.*, vol. 28, no. 3, Mar. 2019, Art. no. 1930003, doi: [10.1142/S0218126619300034](https://doi.org/10.1142/S0218126619300034).
- [5] (2024). *Advanced Encryption Standard (AES)*. Accessed: Jul. 16, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/197/final>
- [6] I. K. Dutta, B. Ghosh, and M. Bayoumi, "Lightweight cryptography for Internet of Insecure Things: A survey," in *Proc. IEEE 9th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2019, pp. 0475–0481.
- [7] M. A. Jimal, M. R. Z'aba, M. L. B. M. Kiah, M. Y. I. Idris, N. Jamil, M. S. Mohamad, and M. S. Rohmad, "Authenticated encryption schemes: A systematic review," *IEEE Access*, vol. 10, pp. 14739–14766, 2022.
- [8] A. Bushnag, "A wireless ECG monitoring and analysis system using the IoT cloud," *Intell. Autom. Soft Comput.*, vol. 33, no. 1, pp. 51–70, 2022.
- [9] A. M. Alwakeel, "An overview of fog computing and edge computing security and privacy issues," *Sensors*, vol. 21, no. 24, p. 8226, Dec. 2021.
- [10] S. Khan, W.-K. Lee, and S. O. Hwang, "Scalable and efficient hardware architectures for authenticated encryption in IoT applications," *IEEE Internet Things J.*, vol. 8, no. 14, pp. 11260–11275, Jul. 2021.
- [11] (2024). *Lightweight Cryptography*. Accessed: Jul. 11, 2024. [Online]. Available: <https://csrc.nist.gov/projects/lightweight-cryptography>
- [12] (2024). *Lightweight Cryptography*. Accessed: Jul. 11, 2024. [Online]. Available: <https://csrc.nist.gov/Projects/lightweight-cryptography/round-1-candidates>
- [13] (2024). *Lightweight Cryptography*. Accessed: Jul. 11, 2024. [Online]. Available: <https://csrc.nist.gov/Projects/lightweight-cryptography/round-2-candidates>
- [14] (2024). *Lightweight Cryptography*. Accessed: Jul. 11, 2024. [Online]. Available: <https://csrc.nist.gov/Projects/lightweight-cryptography/finalists>
- [15] (2023). *Lightweight Cryptography Standardization Process: Nist Selects Ascon*. Accessed: Jul. 11, 2024. [Online]. Available: <https://csrc.nist.gov/news/2023/lightweight-cryptography-nist-selects-ascon>
- [16] (2024). *CAESAR Submissions*. Accessed: Jul. 18, 2024. [Online]. Available: <https://competitions.cr.yt/caesar-submissions.html>
- [17] Z. Zhu, J. Zhang, J. Zhao, J. Cao, D. Zhao, G. Jia, and Q. Meng, "A hardware and software task-scheduling framework based on CPU+FPGA heterogeneous architecture in edge computing," *IEEE Access*, vol. 7, pp. 148975–148988, 2019.
- [18] P. Yalla and J.-P. Kaps, "Evaluation of the Caesar hardware api for lightweight implementations," in *Proc. Int. Conf. ReConFigurable Comput. FPGAs*, 2017, pp. 1–6.
- [19] W. Diehl, F. Farahmand, A. Abdulgadir, J.-P. Kaps, and K. Gaj, "Face-off between the Caesar lightweight finalists: ACORN vs. Ascon," in *Proc. Int. Conf. Field-Programmable Technol. (FPT)*, Dec. 2018, pp. 330–333.
- [20] S. Khan, W.-K. Lee, and S. O. Hwang, "Evaluating the performance of ascon lightweight authenticated encryption for AI-enabled IoT devices," in *Proc. TRON Symp. (TRONSHOW)*, Dec. 2022, pp. 1–6.
- [21] H. Groß, E. Wenger, C. Dobraunig, and C. Ehrenhöfer, "Suit up!—Made-to-measure hardware implementations of ASCON," in *Proc. Euromicro Conf. Digit. Syst. Design*, Aug. 2015, pp. 645–652.
- [22] H. Gross, E. Wenger, C. Dobraunig, and C. Ehrenhöfer, "Ascon hardware implementations and side-channel evaluation," *Microprocessors Microsystems*, vol. 52, pp. 470–479, Jul. 2017.
- [23] N. Roussel, O. Potin, G. Di Pendina, J.-M. Dutertre, and J.-B. Rigaud, "CMOS/STT-MRAM based ascon LWC: A power efficient hardware implementation," in *Proc. 29th IEEE Int. Conf. Electron., Circuits Syst. (ICECS)*, Oct. 2022, pp. 1–4.
- [24] B. Rezvani, F. Coleman, S. Sachin, and W. Diehl. (2019). *Hardware Implementations of Nist Lightweight Cryptographic Candidates: A First Look*. Accessed: Jul. 19, 2024. [Online]. Available: <https://eprint.iacr.org/2019/824.pdf>
- [25] K. Raj and S. Bodapati, "FPGA based light weight encryption of medical data for IoMT devices using ASCON cipher," in *Proc. IEEE Int. Symp. Smart Electron. Syst. (iSES)*, Dec. 2022, pp. 196–201.
- [26] S.-N. Tran, V.-T. Hoang, and D.-H. Bui, "A hardware architecture of NIST lightweight cryptography applied in IPSec to secure high-throughput low-latency IoT networks," *IEEE Access*, vol. 11, pp. 89240–89248, 2023.
- [27] S. Khan, K. Inayat, F. B. Muslim, Y. A. Shah, M. Atif Ur Rehman, A. Khalid, M. Imran, and A. Abdusalomov, "Securing the IoT ecosystem: ASIC-based hardware realization of ascon lightweight cipher," *Int. J. Inf. Secur.*, vol. 1, pp. 1–26, Aug. 2024, doi: [10.1007/S10207-024-00904-1](https://doi.org/10.1007/S10207-024-00904-1).
- [28] A. Kandi, A. Baksi, T. Gerlich, S. Guilley, P. Gan, J. Breier, A. Chattopadhyay, R. R. Shrivastwa, Z. Martinasek, and S. Bhasin. (2024). *Hardware Implementation of ASCON*. Accessed: Jul. 21, 2024. [Online]. Available: <https://csrc.nist.gov/csrc/media/Events/2023/lightweight-cryptography-workshop-2023/documents/accepted-papers/07-hardware-implementation-of-ascon.pdf>
- [29] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," *J. Cryptol.*, vol. 34, no. 3, pp. 1–42, Jul. 2021.
- [30] M. Imran, F. Almeida, J. Raik, A. Basso, S. S. Roy, and S. Pagliarini, "Design space exploration of SABER in 65nm ASIC," in *Proc. 5th Workshop Attacks Solutions Hardw. Secur.*, Nov. 2021, pp. 85–90, doi: [10.1145/3474376.3487278](https://doi.org/10.1145/3474376.3487278).
- [31] T.-H. Nguyen, B. Kieu-Do-Nguyen, C.-K. Pham, and T.-T. Hoang, "High-speed NTT accelerator for CRYSTAL-kyber and CRYSTAL-dilithium," *IEEE Access*, vol. 12, pp. 34918–34930, 2024.
- [32] A. Kandi, A. Baksi, P. Gan, S. Guilley, T. Gerlich, J. Breier, A. Chattopadhyay, R. R. Shrivastwa, and S. Bhasin. (2024). *Side-Channel and Fault Resistant ASCON Implementation: A Detailed Hardware Evaluation*. [Online]. Available: <https://eprint.iacr.org/2024/984>
- [33] W. Diehl, A. Abdulgadir, F. Farahmand, J.-P. Kaps, and K. Gaj, "Comparison of cost of protection against differential power analysis of selected authenticated ciphers," in *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust (HOST)*, Apr. 2018, pp. 147–152.



Tabuk, Saudi Arabia, since 2009. He acquired several academic certificates and published many scientific articles. His research interests include mobile and smart device applications, biometric, security, networking, and machine learning techniques.



he was a Senior Research Member with the Cybersecurity Laboratory, Colorado University. His research interests include software-defined networking, artificial intelligence, cloud computing, the IoT, and cybersecurity. He received multiple research awards from UCCS, UT, and SACM for his outstanding research articles.

ADEL R. ALHARBI received the Bachelor of Science degree in computer science from Qassim University, Saudi Arabia, in 2008, and the dual Master of Science degree in security engineering and computer engineering and the Doctor of Philosophy degree in computer engineering from Southern Methodist University, Dallas, TX, USA, in 2013, 2015, and 2017, respectively. He has been a Faculty Staff Member with the College of Computing and Information Technology, University of

AMER ALJAEDI received the B.Sc. degree from King Saud University, Saudi Arabia, in 2007, the M.Sc. degree in information systems security from Concordia University of Edmonton, Canada, in 2011, and the Ph.D. degree in security engineering from the Computer Science Department, Colorado University, Colorado Springs, USA, in 2018. He is currently an Associate Professor with the College of Computing and Information Technology, University of Tabuk. Before that,



interests include HPC, cloud computing, power efficiency, and the IoT.

ABDULLAH ALJUHNI received the B.Sc. degree in computer science from Taibah University, Saudi Arabia, in 2005, and the M.Sc. degree in computer science and the Ph.D. degree in computer science engineering from the Computer Science Department, University of Colorado, Colorado Springs, USA, in 2016 and 2021, respectively. Currently, he is an Assistant Professor with the College of Computing and Information Technology, University of Tabuk. His research



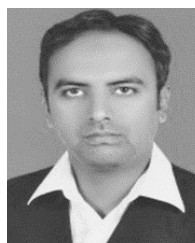
of Newcastle, and GulfNet Solutions Company. He is a member of several cybersecurity committees and institutions, including the World Economic Forum and the United Nations, and a recipient of many prestigious international honors and awards, he has published multiple research articles. He holds numerous certifications, including CISM, CISA, PMP, DRP, Security+, ISO 27001, ISO 27005, and ECIH.

HUSSAIN ALDAWOOD received the bachelor's degree in management information systems from The University of Arizona, the M.B.A. degree from Florida Atlantic University, and the Ph.D. degree in cybersecurity from The University of Newcastle. He is currently the Director with NEOM. He is a distinguished Cybersecurity Subject Matter Expert with 14 years of experience in critical infrastructure protection. His career includes roles at Saudi Aramco, The University



transportation safety, he has authored several research articles concerning smart cities and the Internet of Things. His research interests include intelligent transportation systems, advanced logistics technologies, artificial intelligence, transportation data analytics, and applications related to transportation safety. He is a member of several committees at the university.

MOAHD K. ALGHUSON received the bachelor's degree from Wilkes University, Wilkes-Barre, PA, USA, in 2011, and the master's and Ph.D. degrees from Southern Methodist University, Dallas, TX, USA, in 2013 and 2020, respectively. He is currently an Assistant Professor with the Faculty of Engineering, Department of Industrial Engineering, University of Tabuk. He is the Vice Dean of research and graduate studies with the University of Tabuk. In addition to traffic and



SAJJAD SHAUKAT JAMAL received the Ph.D. degree in mathematics from Quaid-i-Azam University, Islamabad, Pakistan. Currently, he is an Assistant Professor with the Department of Mathematics, King Khalid University, Abha, Saudi Arabia. He has a number of quality research articles in well-reputed journals on application of mathematics in multimedia security. His research interests include mathematics, number theory, cryptography, digital watermarking, and steganography.

...